CHAPTER 1

Introduction

Initial Terminology

In the contemporary world, daily interaction with databases is a fact of life. It occurs in many everyday situations, such as when we make purchases, view content on the internet, or engage in banking transactions. Databases are vital for the functioning of modern companies and organizations. This book will provide coverage of the most fundamental issues and topics related to the development and use of databases and database systems. We will start with an overview of the basic database-related terms.

The term **data** refers to facts that are recorded and can be accessed. In addition to text and numbers, data can also appear in other formats, such as figures, graphics, images, and audio/video recordings. Whatever the format is, the data is recorded and kept because it is considered to be of use to an intended user.

The term **information** refers to the data that is accessed by a user for some particular purpose. Typically, getting the needed information from a collection of data requires performing an activity, such as searching through, processing, or manipulating the data in some form or fashion.

To illustrate the terms "data" and "information," let us consider a phone book. A phone book is a collection of data—the phone numbers of people, restaurants, banks, and so on. Within the appropriate context, these phone numbers become information. For example, if a person in possession of a phone book becomes hungry and decides to make a phone call and have a pizza delivered to his or her house from the nearest pizza delivery place (e.g., Pizza Adria), the phone number of Pizza Adria becomes the information that this person needs. Searching for and finding the phone number of Pizza Adria in the collection of data in the phone book is the necessary action that will yield the information from the data.

In another example, let us assume that a manager in a large retail company, ZAGI, needs information about how good the sales of apparel in the current quarter are. Let us assume that every single sales transaction within this retail company is recorded in a computer file available to the manager. Even though such data captures the needed information about the apparel's sales performance, examination of each individual record in such an enormous file by the manager is not a feasible method for producing the needed information. Instead, the data has to be processed in a way that can give insight to the manager. For example, the data about apparel sales in the current quarter can be summarized. This summary can then be compared to the summarized sales of apparel in the previous quarter or to the summarized sales of apparel in the same quarter from last year. Such actions would yield the needed information from this collection of data.

The terms "data" and "information" are often interchanged and used as synonyms for each other. Such practice is very common and is not necessarily wrong. As we just explained, information is simply the data that we need. If the data that an organization gathers and stores has a purpose and satisfies a user's need, then such data is also information.[1] In this

[1] On the other hand, there are situations when the portion of the stored data that is of little or no use in the organization is so significant that it obscures the data that is actually useful. This is often referred to as a "too much data, too little information" scenario.

111	Joe	45
123	Sue	17
101	Bob	55
341	Joe	74
117	Pam	101

Figure 1.1 Data without metadata

book, we will frequently use the terms "data" and "information" as interchangeable synonyms.

Metadata is the data that describes the structure and the properties of the data. Metadata is essential for the proper understanding and use of the data. Consider a data sample shown in Figure 1.1. It is difficult to make sense of the data shown in Figure 1.1 without the understanding of its metadata. The columns of numeric and textual data as presented in Figure 1.1 have no meaning.

Clients in Default		
ClientID	**ClientName**	**DaysOverdue**
111	Joe	45
123	Sue	17
101	Bob	55
341	Joe	74
117	Pam	101

Figure 1.2 Data with metadata

The data from Figure 1.1 is shown again in Figure 1.2, now accompanied by its metadata. It is now clear what the meaning of the shown data is. The metadata explains that each row represents a client who is in default. The first column represents a client identifier, the second column refers to the client name, and the third column shows how many days the client is overdue.

A **database** is a structured collection of related data stored on a computer medium. The purpose of a database is to organize the data in a way that facilitates straightforward access to the information captured in the data. The structure of the database is represented in the database metadata. The **database metadata** is often defined as the data about the data or the database content that is not the data itself. The database metadata contains the following information:

- names of data structures (e.g., names of tables, names of columns);
- data types (e.g., column ClientName—data type Character; column DaysOverdue —data type Integer);
- data descriptions (e.g., "column DaysOverdue represents the number of days that the client is late in making their payment");
- other information describing the characteristics of the data that is being stored in a database.

A **database management system (DBMS)** is software used for the following purposes:

- creation of databases;
- insertion, storage, retrieval, update, and deletion of the data in the database;
- maintenance of databases.

For example, the relationship between a DBMS and a database is similar to the relationship between the presentation software (such as MS PowerPoint) and a presentation. Presentation software is used to create a presentation, insert content in a presentation, conduct a presentation, and change or delete content in a presentation.

Similarly, a DBMS is used to create a database, to insert the data in the database, to retrieve the data from the database, and to change or delete the data in the database.

A **database system** is a computer-based system whose purpose is to enable an efficient interaction between the users and the information captured in a database. A typical architecture is shown in Figure 1.3.

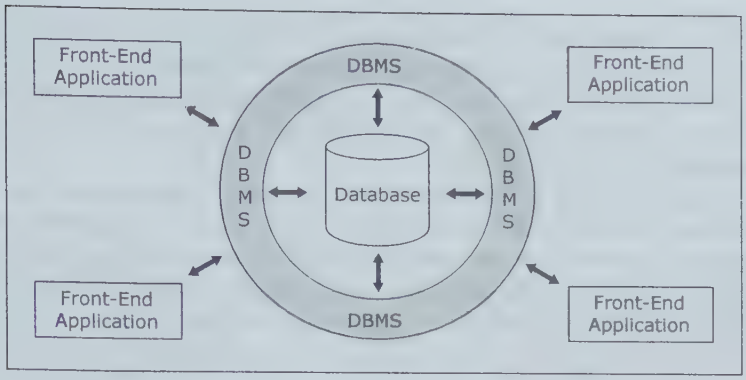

Figure 1.3 Typical database system architecture

The three main components of a database system[2] are the database, the DBMS, and the front-end applications. At the heart of a database system is the database. All interaction with the database occurs via the DBMS. **Front-end applications** are created in order to provide a mechanism for easy interaction between the users and the DBMS. Consider the following example of a user interacting with the DBMS by using a front-end application.

An ATM screen showing options to the user, such as "Withdrawal from checking account" or "Withdrawal from savings account," is an example of a front-end application. When a user makes a choice by selecting from the options on the screen, a communication between the front-end application and the DBMS is triggered, which subsequently causes a communication between the DBMS and the database. For instance, selecting from the ATM screen options, a user can request a withdrawal of $20 from the checking account. As a result, the front-end application issues several commands to the DBMS on the user's behalf. One command tells the DBMS to verify in the database if the user has enough money in the checking account to make the withdrawal. If there is enough money, the ATM releases the money to the user, while another command is issued to the DBMS to reduce the value representing the amount of money in the user's checking account in the database by $20.

Users employing a database system to support their work- or life-related tasks and processes are often referred to as **end users (business users)**. The term "end user" distinguishes the actual users of the data in the database system from the technical personnel engaging in test-use during the implementation and maintenance of database systems.

The type of interaction between the end user and the database that involves front-end applications is called **indirect interaction**. Another type of interaction between the end users and the database, called **direct interaction**, involves the end user directly communicating with the DBMS. Figure 1.4 illustrates direct and indirect interaction.

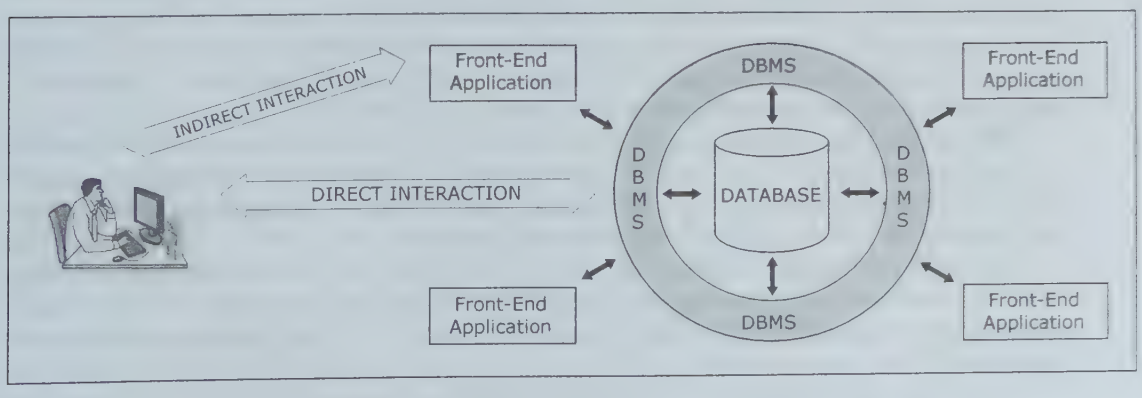

Figure 1.4 Direct and indirect interaction between the end user and the database

[2] Another commonly used term for the system whose architecture is shown in Figure 1.3 is "information system." The terms "information system" and "database system" often have the same meaning, but they are used in different contexts. Since databases are the focus of this book, we will use the synonym "database system."

Whereas indirect interaction typically requires very little or no database skill from the end user, the level of expertise and knowledge needed for direct interaction requires database-related training of the end users. The direct interaction requires that the end user knows how to issue commands to the specific DBMS. This typically requires that the end user knows the language of the DBMS.

The database and DBMS are mandatory components of a database system. In an environment where every person who needs access to the data in the database is capable of direct interaction and has the time for it, the front-end component is not necessary and the database system can function without it. However, in the vast majority of cases in everyday life and business, database systems have a front-end component, and most of the interactions between the end users and the database occur via front-end applications as indirect interactions.

Steps in the Development of Database Systems

Once the decision to undertake the database project is made and the project is initiated, the predevelopment activities, such as planning and budgeting, are undertaken. Those activities are followed by the actual process of the development of the database system. The principal activities in the process of the development of the database system are illustrated in Figure 1.5.

Next, we give a brief discussion and description of each of the steps shown in Figure 1.5.

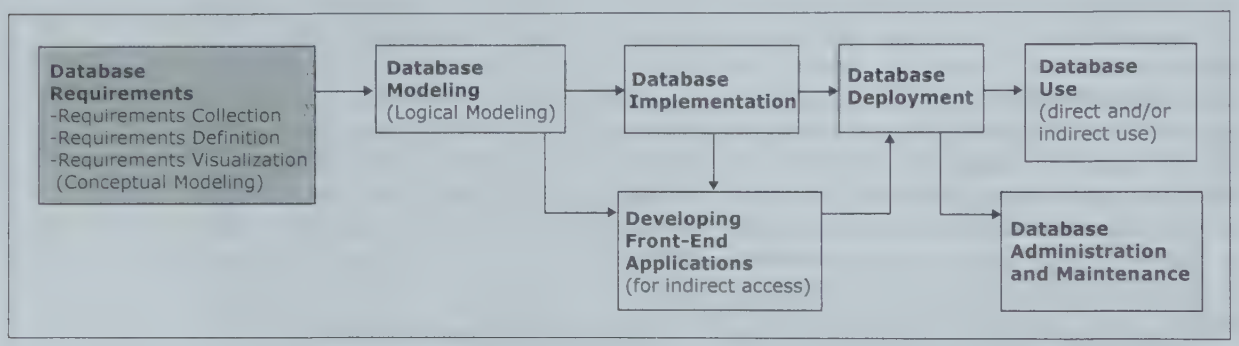

Figure 1.5 Steps in the development of database systems

Database Requirements Collection, Definition, and Visualization

The first and most critical step in the development of the database is **requirements collection, definition, and visualization**. If this step is successful, the remaining steps have a great chance of success. However, if this step is done incorrectly, all of the remaining steps, and consequently the entire project, will be futile. In Figures 1.5 and 1.6 we highlight this step with a gray background in order to underscore its critical nature.

This step results in the end user requirements specifying which data the future database system will hold and in what fashion, and what the capabilities and functionalities of the database system will be. The requirements are used to model and implement the database and to create the front-end applications for the database.

The collected requirements should be clearly defined and stated in a written document, and then visualized as a conceptual database model by using a conceptual data modeling technique, such as entity-relationship (ER) modeling.[3] "Conceptual data modeling" is another term for requirements visualization.

The guidelines and methods for database requirements phase call for an iterative process. A smaller beginning set of requirements can be collected, defined, and visualized, and then discussed by the database developers and intended end users. These discussions can

[3] Entity-relationship (ER) modeling is covered in the next chapter.

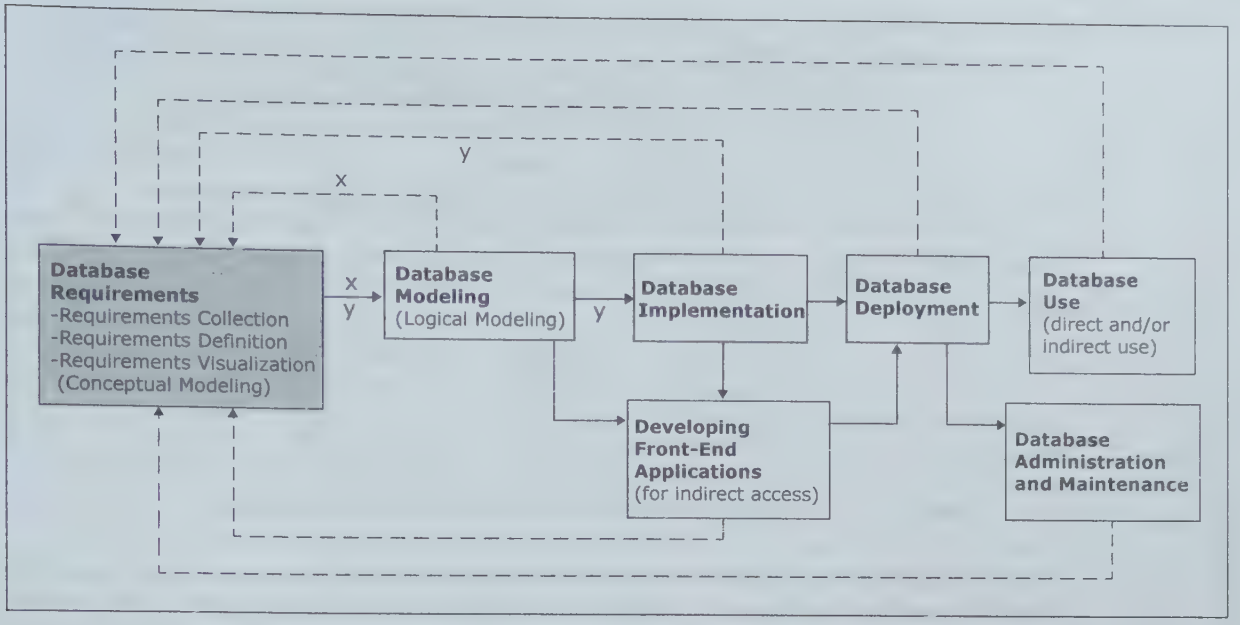

Figure 1.6 Iterative nature of the database requirements collection, definition, and visualization process

then lead into another iteration of collecting, defining, and visualizing requirements that gradually increases the first set of requirements.

Even when a set of requirements is agreed upon within the database requirements collection, definition, and visualization step, it can still be subject to change initiated by the other steps in the database development process, as illustrated by Figure 1.6.

Instead of mandating the collection, definition, and visualization of all database requirements in one isolated process, followed by all other steps in the development of the database systems, the common recommendation is to allow refining and adding to the requirements after each step of the database development process. This is illustrated by the dashed lines in Figure 1.6.

For example, a common practice in database projects is to collect, define, and visualize an initial partial set of requirements and, based on these requirements, create and implement a preliminary partial database model. This is followed by a series of similar iterations (marked by the letter *x* in Figure 1.6), where the additional requirements are added (collected, defined, and visualized) and then used to expand the database model.

The requirements can also be altered iteratively when other steps, such as the creation of front-end applications or actual database use, reveal the need for modifying, augmenting, or reducing the initial set of requirements.

Every time the set of requirements is changed, the conceptual model has to be changed accordingly, and the changes in the requirements must propagate as applicable through all of the subsequent steps: modeling, creating the database, creating front-end applications, deployment, use, and administration/maintenance.

No implicit changes of requirements are permitted in any of the database development steps. For example, during the database implementation process (middle rectangle in Figure 1.6) a developer is not allowed to create in an ad hoc way a new database construct (e.g., a database table or a column of a database table) that is not called for by the requirements. Instead, if during the database implementation process it is discovered that a new construct is actually needed and useful, the proper course of action (marked by the letter y in Figure 1.6) is to go back to the requirements and augment both the requirements document and the conceptual model to include the requirement for the new construct. This new requirement should then be reflected in the subsequently augmented database model. Only then should a new database construct actually be implemented.

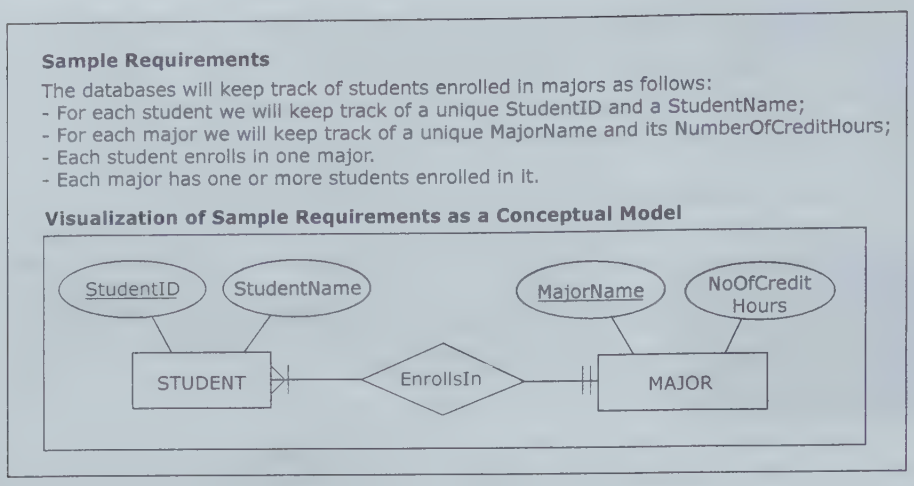

Figure 1.7 Sample database requirements and conceptual database model

Forming database requirements is widely recognized as the most critical step in the database system development process. The outcome of this step determines the success of the entire database project. If this step is not done correctly, the requirements will be off-target and, consequently, the resulting database will not properly satisfy the needs of its end users.

Figure 1.7 shows a small example of requirements that were collected, defined (written down), and visualized as a conceptual model using ER modeling technique. This figure is for illustration purposes only, and the details of requirement collection and conceptual modeling process will be covered in Chapter 2.

Database Modeling

The first step following requirements collection, definition, and visualization is **database modeling (logical database modeling)**. In this book, we use the term "database modeling" to refer to the creation of the database model that is implementable by the DBMS software. Such a database model is also known as a **logical database model (implementational database model)**, as opposed to a conceptual database model. A conceptual database model is simply a visualization of the requirements, independent of the logic on which a particular DBMS is based. Therefore, a database has two models: a conceptual model created as a visualization of requirements during the requirements collection, definition, and visualization step; and an actual database model, also known as a logical model, created during the database modeling step to be used in the subsequent step of database implementation using the DBMS. The conceptual model serves as a blueprint for the actual (logical) database model.

In most modern databases, database modeling (i.e., logical database modeling) involves the creation of the so-called relational database model.[4] When ER modeling is used as a conceptual modeling technique for visualizing the requirements, relational database modeling is a straightforward process of mapping (converting) the ER model into a relational model.

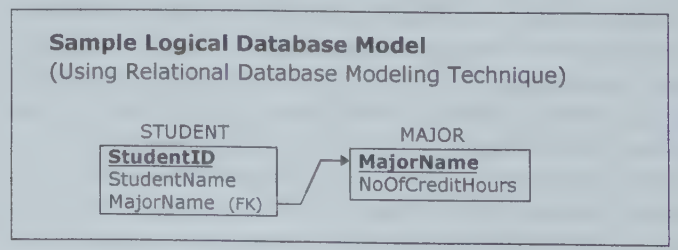

Figure 1.8 Sample logical database model

[4] The relational database model is covered in Chapters 3 and 4.

Figure 1.8 shows a small example of a logical database model, converted from the conceptual model in Figure 1.7. This figure is for illustration purposes only, and the details of logical database modeling will be covered in Chapter 3.

Database Implementation

Once the database model is developed, the next step is database implementation. This step involves using a DBMS to implement the database model as an actual database that is initially empty. Database implementation is a straightforward process that involves database developers using the DBMS functionalities and capabilities to implement the database model as an actual functioning database, much in the same way a construction crew uses construction equipment to implement a blueprint for a building as an actual building. As we mentioned, most modern databases are modeled as relational databases, and for that reason they are implemented using relational DBMS (RDBMS) software. SQL (Structured Query Language)[5] is a language used by most relational DBMS software packages. Among its features, SQL includes commands for creating, modifying, and deleting database structures. These commands are used during database implementation.

Figure 1.9 shows a small example of a SQL code that creates the sample database based on the model shown in Figure 1.8. This figure is for illustration purposes only, and the details of SQL will be covered in Chapter 5.

Sample SQL Code for Creating Databases

```
CREATE TABLE MAJOR
(
   MajorName CHAR(20),
   NoOfCreditHours INT,
   PRIMARY KEY (MajorName)
);

CREATE TABLE STUDENT
(
   StudentID INT,
   StudentName CHAR(10),
   MajorName CHAR(20),
   PRIMARY KEY (StudentID),
   FOREIGN KEY (MajorName) REFERENCES MAJOR(MajorName)
);
```

Figure 1.9 Sample SQL code for creating a database

Developing Front-End Applications

The process of developing front-end applications refers to designing and creating applications for indirect use by the end users. Such applications are included in most database systems. The front-end applications are based on the database model and the requirements specifying the front-end functionalities of the system needed by the end users. Front-end applications usually contain interfaces, such as forms and reports, accessible via navigation mechanisms, such as menus. Figure 1.5 illustrates that the design and creation of front-end applications can commence and proceed in parallel with database implementation. For example, the look and feel of the front-end applications, as well as the number of particular components (e.g., forms and reports) and their individual functionalities, can be determined before (or while) the database is implemented. Of course, the actual creation of a front-end application also involves connecting it to the database. Connecting a front-end application to the database can only be done once the database is implemented.

[5] SQL is also sometimes referred to as "Sequel." SQL is covered in Chapter 5.

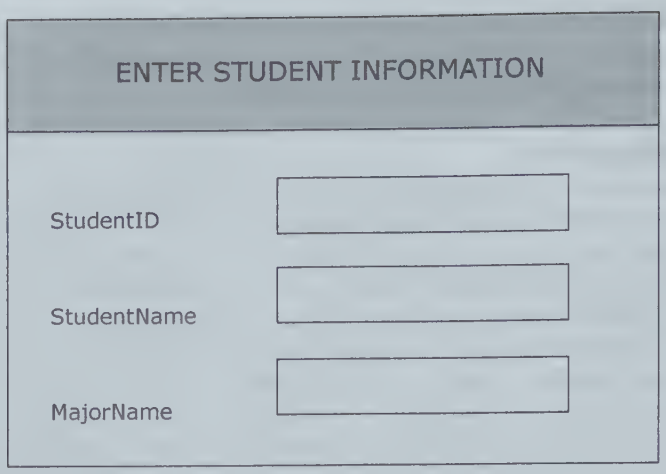

Figure 1.10 Database front-end example

Figure 1.10 shows a small example of a front-end component for the database created in Figure 1.9. This figure is for illustration purposes only, and the details of database front end will be covered in Chapter 6.

Database Deployment

Once the database and its associated front-end applications are implemented, the next step is **database deployment**. This step involves releasing the database system (i.e., the database and its front-end applications) for use by the end users. Typically, this step also involves populating the implemented database with the initial set of data.

Database Use

Once the database system is deployed, end users engage in **database use**. Database use involves the **insertion, modification, deletion, and retrieval** of the data contained within the database system. The database system can be used indirectly, via the front-end applications, or directly via the DBMS. As we mentioned, SQL is the language used by most modern relational DBMS packages. SQL includes commands for insertion, modification, deletion, and retrieval of the data. These commands can be issued by front-end applications (indirect use) or directly by the end users themselves (direct use).

Figure 1.11 shows, for illustration purposes, a small example of data for the database created in Figure 1.9. Such data can be inserted during the deployment and it can also be inserted, modified, deleted, or retrieved by the end users as part of the database use.

MAJOR

MajorName	NoOfCreditHours
Accounting	152
IS	138
Marketing	138

STUDENT

StudentID	StudentName	MajorName
111	Kirsten	Marketing
222	Eve	Accounting
333	Zoe	IS
444	Courtney	Marketing
555	Ben	Accounting
666	Finola	IS
777	Iris	Marketing

Figure 1.11 Example of data in a database

Database Administration and Maintenance

The **database administration and maintenance** activities support the end users. Database administration and maintenance activities include dealing with technical issues, such as providing security for the information contained in the database, ensuring sufficient hard-drive space for the database content, and implementing backup and recovery procedures.

The Next Version of the Database

In most cases, after a certain period of use, the need for modifications and expansion of the existing database system becomes apparent, and the development of a new version of the existing database system is initiated. The new version of the database should be created following the same development steps as the initial version.

As with the initial version of the database system, the development of subsequent versions of the database system will start with the requirements collection, definition, and visualization step. Unlike with the initial version, in the subsequent versions not all requirements will be collected from scratch. Original requirements provide the starting point for additions and alterations. Many of the additions and modifications result from observations and feedback by the end users during the use of the previous version, indicating the ways in which the database system can be improved or expanded. Other new requirements may stem from changes in the business processes that the database system supports, or changes in underlying technology. Whatever the reasons are, change is inherent for database systems. New versions should be expected periodically and handled accordingly.

Database Scope

Databases vary in their scope, from small single-user (personal) databases to large enterprise databases that can be used by thousands of end users. Regardless of their scope, all databases go through the same fundamental development steps, such as requirements collection, modeling, implementation, deployment, and use. The difference in the scope of databases is reflected in the size, complexity, and cost in time and resources required for each of the steps.

People Involved With Database Systems

The creation and use of database systems involve people in different types of roles. The roles and titles given to the people involved in the creation, maintenance, and use of databases can vary from project to project and from company to company. There are a number of ways in which these roles could be classified. Here, we divide the roles of people involved with the database projects and systems into the following four general categories:

- database analysts, designers, and developers;
- front-end applications analysts and developers;
- database administrators; and
- database end users.

These categories reflect in which part of the database life cycle the different people are involved and in what capacity.

Database Analysts, Designers, and Developers

Database analysts are involved in the requirements collection, definition, and visualization stage. **Database designers (database modelers or architects)** are involved in the database modeling stage. **Database developers** are in charge of implementing the database model as a functioning database using the DBMS software. It is not uncommon for the same people

to perform more than one of these roles.[6] These roles are related to converting business requirements into the structure of a database that is at the center of the database system. An ideal characteristic of the people in these roles is technical and business versatility or, more specifically, the knowledge of database modeling methodologies and of business and organizational processes and principles.

Front-End Applications Analysts and Developers

Front-end applications analysts are in charge of collecting and defining requirements for front-end applications, while **front-end applications developers** are in charge of creating the front-end applications. The role of front-end applications analysts involves finding out which front-end applications would be the most useful and appropriate for indirect use by the intended end users. This role also involves determining the features, look, and feel of each front-end application. The role of the front-end applications developer is to create the front-end applications, based on the requirements defined by the front-end applications analysts. The role of the front-end applications developer often requires knowledge of programming.

Database Administrators

Once the database and the associated front-end applications are designed, implemented, and deployed as a functioning database system, the database system has to be administered. Database administration encompasses the tasks related to the maintenance and supervision of a database system. The role of **database administrator (DBA)** involves managing technical issues related to security (e.g., creating and issuing user names and passwords, granting privileges to access data, etc.), backup and recovery of database systems, monitoring use and adding storage space as needed, and other tasks related to the proper technical functioning of the database system. A database administrator should have the technical skills needed for these tasks, as well as detailed knowledge of the design of the database at the center of the database system.

Database End Users

Arguably, database end users are the most important category of people involved with database systems. They are the reason for the existence of database systems. The quality of a database system is measured by how quickly and easily it can provide the accurate and complete information needed by its end users. End users vary in their level of technical sophistication, in the amount of data that they need, in the frequency with which they access the database system, and in other measurements. As we mentioned before, most end users access the database in an indirect fashion, via the front-end applications. Certain end users that are familiar with the workings of the DBMS and are authorized to access the DBMS can engage in the direct use of the database system.

The four categories we highlighted here represent the roles of people who are directly involved with the database system within an organization or company. In addition to these four categories, other people, such as the developers of the DBMS software or the system administrators in charge of the computer system on which the database system resides, are also necessary for the functioning of database systems.

Operational Versus Analytical Databases

Information that is collected in database systems can be used, in general, for two purposes: an operational purpose and an analytical purpose.

The term **operational information (transactional information)** refers to the information collected and used in support of the day-to-day operational needs in businesses and other organizations. Any information resulting from an individual transaction, such as

[6] In fact (especially in smaller companies and organizations), the same people may be in charge of all aspects of the database system, including design, implementation, administration, and maintenance.

performing an ATM withdrawal or purchasing an airline ticket, is operational information. That is why operational information is also sometimes referred to as "transactional information."

Operational databases collect and present operational information in support of daily operational procedures and processes, such as deducting the correct amount of money from the customer's checking account upon an ATM withdrawal or issuing a correct bill to a customer who purchased an airline ticket.

The term **analytical information** refers to the information collected and used in support of analytical tasks. An example of analytical information is information showing a pattern of use of ATM machines, such as what hours of the day have the most withdrawals, and what hours of the day have the least withdrawals. The discovered patterns can be used to set the stocking schedule for the ATM machines. Another example of analytical information is information revealing sales trends in the airline industry, such as which routes in the United States have the most sales and which routes in the United States have the least sales. This information can be used in planning route schedules.

Note that analytical information is based on operational (transactional) information. For example, to create the analytical information showing a pattern of use of ATM machines at different times of the day, we have to combine numerous instances of transactional information resulting from individual ATM withdrawals. Similarly, to create the analytical information showing sales trends over various routes, we have to combine numerous instances of transactional information resulting from individual airline ticket purchases.

A typical organization maintains and utilizes a number of operational databases. At the same time, an increasing number of companies also create and use separate **analytical databases**. To reflect this reality, this book provides coverage of issues related to both operational and analytical databases. Issues related to the development and use of operational databases are covered in Chapters 2–6, while the topics related to the development and use of analytical databases are covered in Chapters 7–10.

Relational DBMS

The relational database model is the basis for the contemporary DBMS software packages that are used to implement the majority of today's operational and analytical corporate databases. Examples of relational DBMS software include Oracle, MySQL, Microsoft SQL Server, PostgreSQL, IBM DB2, and Teradata. Among these DBMS tools, Teradata was developed specifically to accommodate large analytical databases, whereas the others are used for hosting both operational and analytical databases. In Chapter 10, we will provide an overview of some of the basic functionalities of contemporary relational DBMS packages and illustrate how those functionalities are used for administration and maintenance of both operational and analytical databases.

Book Topics Overview

This book is devoted to the most fundamental issues and topics related to the design, development, and use of operational and analytical databases. These topics and issues are organized as follows.

This chapter (**Chapter 1**) introduced basic terminology, listed and briefly described the steps in the development of database systems, provided an overview of database scope and the roles of people involved with database systems, and defined operational and analytical databases.

Chapter 2 covers the topics of collection and visualization of requirements for operational databases by giving a detailed overview of ER modeling, a conceptual data modeling technique.

Chapter 3 covers the topics of modeling operational databases by giving a detailed overview of the basics of relational modeling, a logical data modeling technique.

Chapter 4 continues the coverage of modeling operational databases, discussing additional topics related to the relational database model, such as database normalization.

Chapter 5 provides detailed coverage of SQL and its functionalities for creating and using relational databases.

Chapter 6 gives an overview of basic topics related to the implementation and use of operational database systems.

Chapter 7 introduces basic terminology and concepts related to analytical databases, which are also known as data warehouses and data marts.

Chapter 8 covers the topics of modeling analytical databases by giving an overview of data warehouse/data mart modeling techniques.

Chapter 9 gives an overview of basic topics related to the implementation and use of data warehouses/data marts.

Chapter 10 gives an overview of big data and data lakes.

Chapter 11 provides a concise overview of DBMS functionalities and the database administration topics that are universal for both operational and analytical databases.

The **appendices** provide brief overviews of selected additional database and database-related issues.

Key Terms

Analytical databases
Analytical information
Conceptual database model
Data
Database
Database administration and maintenance
Database administrator (DBA)
Database analysts
Database deployment
Database designers (database modelers or architects)
Database developers
Database implementation
Database management system (DBMS)
Database metadata
Database modeling (logical database modeling)
Database system
Database use
Developing front-end applications
Direct interaction
End users (business users)
Front-end application
Front-end applications analysts
Front-end applications developers
Indirect interaction
Information
Insertion, modification, deletion and retrieval
Logical Database Models (implementational database models)
Metadata
Operational databases
Operational information (transactional information)
Requirements collection, definition, and visualization (requirements analysis)

Review Questions

Q1.1 Give several examples of instances of data.

Q1.2 Give several examples of converting data to information.

Q1.3 Create your own example that shows a collection of data, first without the metadata and then with the metadata.

Q1.4 Describe the relationship between the database and DBMS.

Q1.5 What are the main components of a database system?

Q1.6 Give an example of an indirect use of a database system.

Q1.7 What are the steps in the development of database system?

Q1.8 Explain the iterative nature of the database requirements collection, definition and visualization process.

Q1.9 What is the purpose of conceptual database modeling?

Q1.10 What is the purpose of the logical database modeling?

Q1.11 Briefly describe the process of database implementation.

Q1.12 Briefly describe the process of developing the front-end applications.

Q1.13 What takes place during the database deployment?

Q1.14 What four data operations constitute the database use?

Q1.15 Give examples of database administration and maintenance activities.

Q1.16 What are the similarities and differences between the development of the initial and subsequent versions of the database?

Q1.17 How does the scope of the database influence the development of the database system?

Q1.18 What are the main four categories of people involved with database projects?

Q1.19 What is the role of database analysts?

Q1.20 What is the role of database designers?

Q1.21 What is the role of database developers?

Q1.22 What is the role of front-end application analysts?

Q1.23 What is the role of front-end application developers?

Q1.24 How is the term "quality of a database system" related to the end users?

Q1.25 Give an example of operational (transactional) information.

Q1.26 Give an example of analytical information.

Q1.27 List several relational DBMS software packages.